

# **Software Design Patterns**

## **Introduction to Software Design Patterns**

Dr. Marwa Ahmed  
Marwa.ahmed@ecu.edu.eg

# Course References

- Primary reference: Eric Freeman and Elisabeth Robson, Head First Design Patterns, 2nd ed., O'Reilly Media, 2020.
- Supplementary reference: Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software (GoF).

# Software Design patterns

- Software development problems are often recurring problems.
- Other developers have already encountered and solved many design problems.
- Design patterns translate accumulated object-oriented experience into reusable designs.
- Patterns provide experience reuse, rather than simply code reuse.
- Software Design patterns are proven ways to organize software to solve common design problems; they give us a design solution, not ready-made code.

# What is Good Software Design?

- **Maintainable:** easy to modify when requirements change.
- **Reusable:** behaviors or components can be reused in other scenarios.
- **Flexible:** new behavior can be added without major restructuring.
- **Extensible:** the system can grow without becoming difficult to manage.
- **Knowing OO:** OO concepts such as inheritance and polymorphism does not automatically produce good design.

# Why Software Design must Handle Change

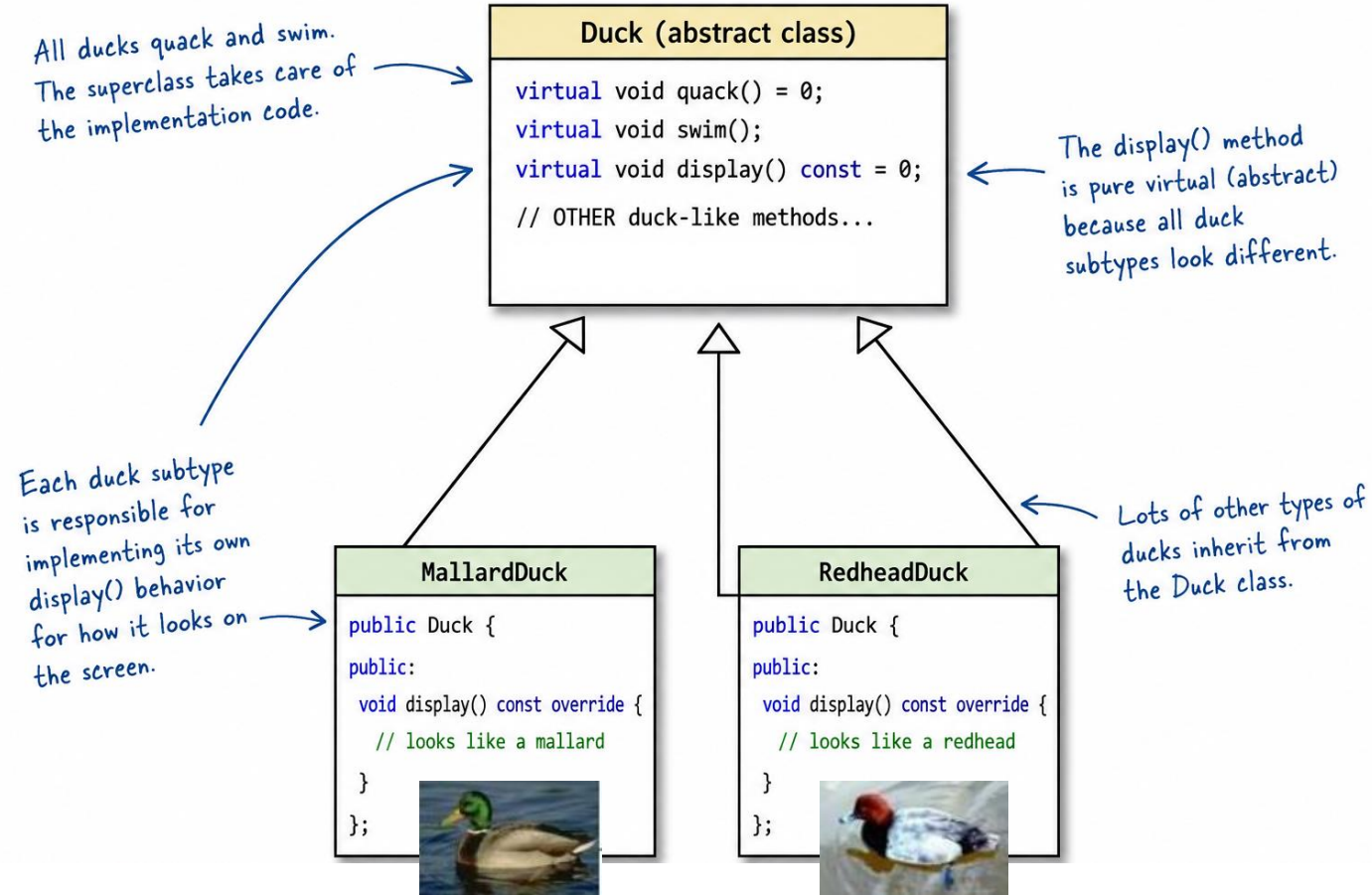
- Change is presented as the constant in software development.
- Users may request new functionality.
- Organizations may change vendors, data formats, protocols, or technologies.
- Good design aims to make changes with the least possible impact on existing code.

# Maintainability

- Maintainability is the ability to modify and update software without widespread unintended effects.
- Poor design can cause:
  - Changes in one class to affect many other classes.
  - Repeated modifications.
  - Duplicate code.
  - Increased chance of bugs.
  - Difficult maintenance.

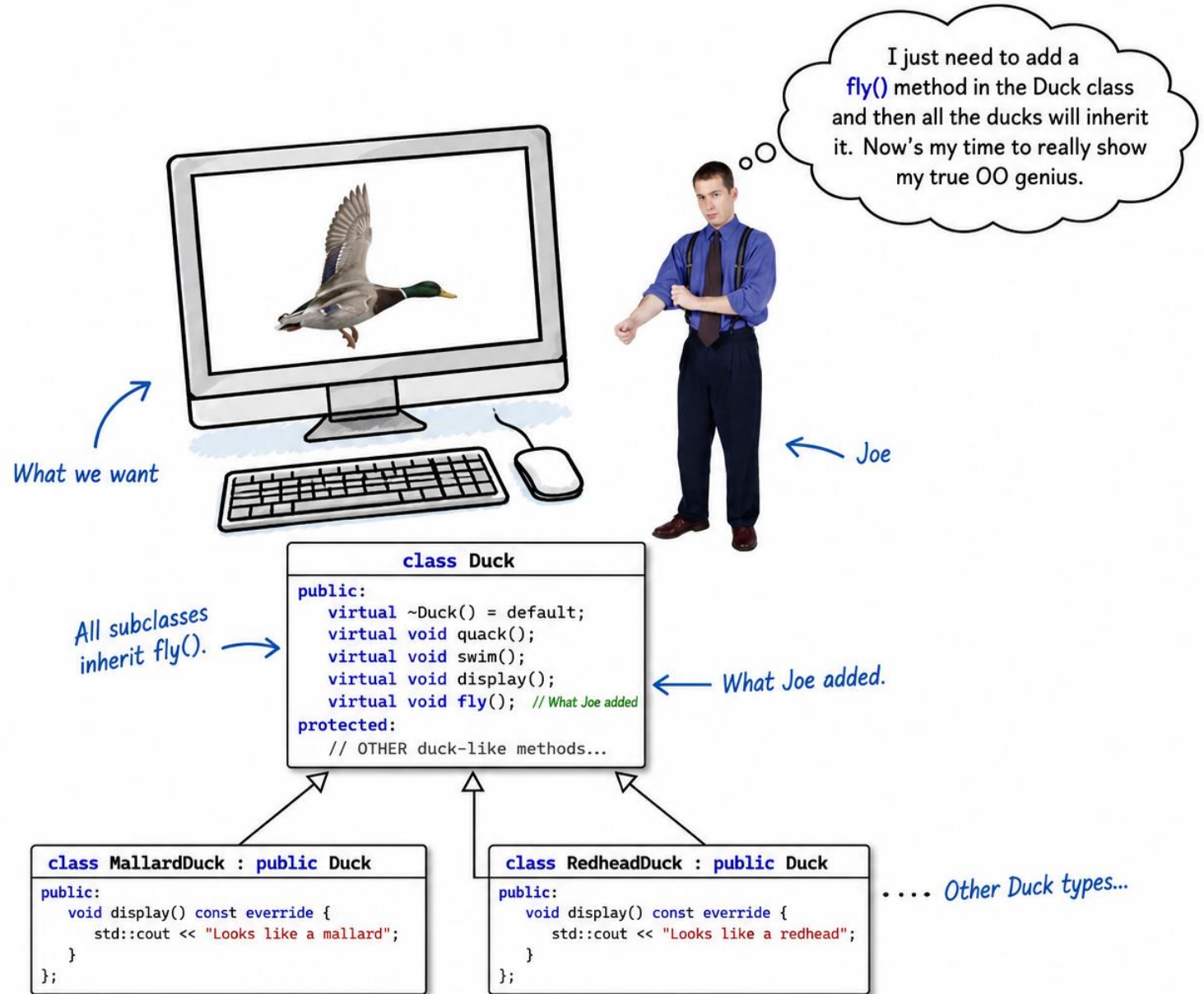
# Example: SimUDuck

- Joe works for a company that makes a highly successful duck pond simulation game, SimUDuck.
- The game can show a wide variety of duck species swimming and making quacking sounds.
- The initial designers of the system used standard OO techniques and created a single superclass called Duck from which all other duck types inherit.



# Example: SimUDuck

- In the last year, the company managers think it's time for a big innovation. They need something really impressive to show at the upcoming shareholders meeting next week.
- The managers decided that flying ducks is just what the simulator needs to outperform the competition. And of course Joe's manager told them it will be no problem for Joe to just whip something up in a week.



# Example: SimUDuck

## Example usage

```
Duck* d1 = new MallardDuck();  
Duck* d2 = new RedheadDuck();  
Duck* d3 = new RubberDuck();
```

```
d1->fly(); // flies  
d2->fly(); // flies  
d3->fly(); // oops! rubber duck flies too!
```

## Problem:

A change in the superclass (adding fly()) caused unintended behavior in subclasses that shouldn't fly.

## Result:

Flying rubber ducks!

Joe, I'm at the shareholders meeting. They just gave a demo and there were rubber duckies flying around the screen. Was this your idea of a joke?

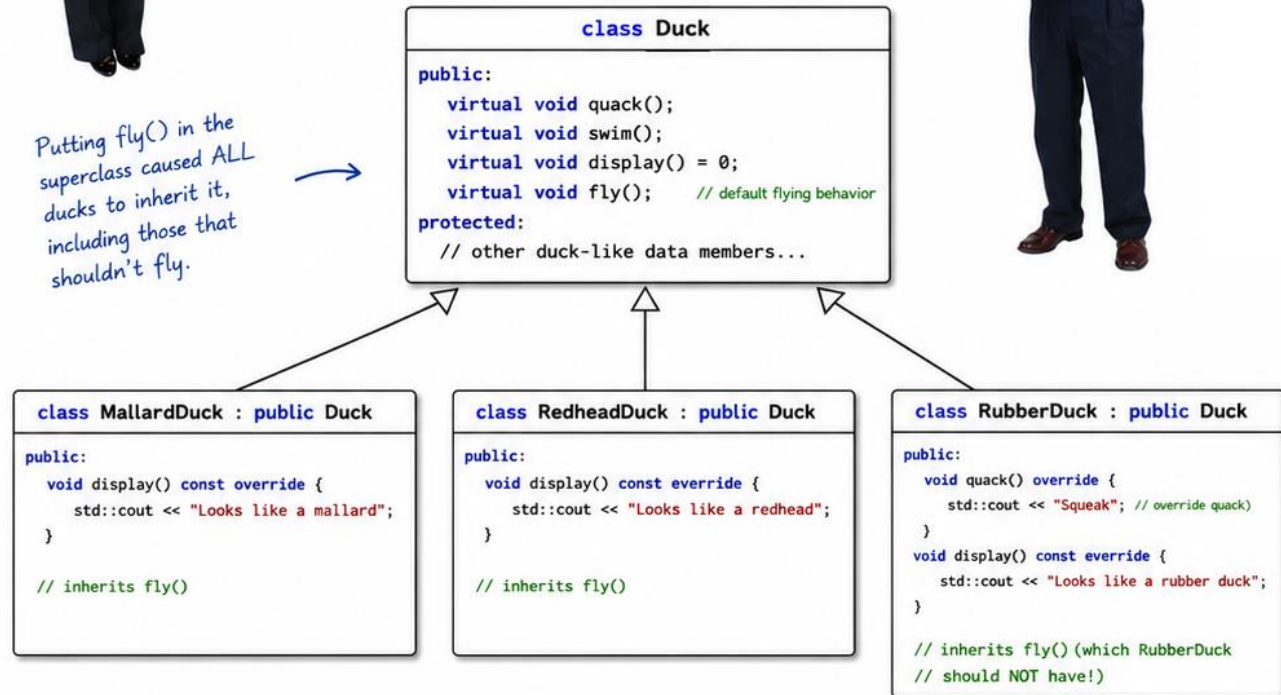
## What happened?

Joe failed to notice that not *all* subclasses of Duck should fly. When Joe added new behavior to the Duck superclass, he was also adding behavior that was *not* appropriate for some Duck subclasses. He now has flying inanimate objects in the SimUDuck program.

*A localized update to the code caused a non-local side effect (flying rubber ducks)!*

Okay, so there's a slight flaw in my design. I don't see why they can't just call it a "feature." It's kind of cute...

Putting fly() in the superclass caused ALL ducks to inherit it, including those that shouldn't fly.



Rubber ducks don't quack, so quack() is overridden to "Squeak".

# Problems with Inheritance

- All subclasses inherit behavior from the superclass.
- Some subclasses may inherit behavior they should not have.
- Behavior may need to be overridden repeatedly.
- Different behaviors make inheritance difficult to maintain because each new type may need special code or changes.
- Changes can unintentionally affect other subclasses.

# Reusability

- Inheritance can provide reuse, but it can also introduce unwanted behavior.
- Not every subclass needs every inherited behavior.
- **Better approach:** Separating behaviors into independent classes allows them to be reused.
- Example: The same flying behavior can be reused by different classes, not just Ducks.

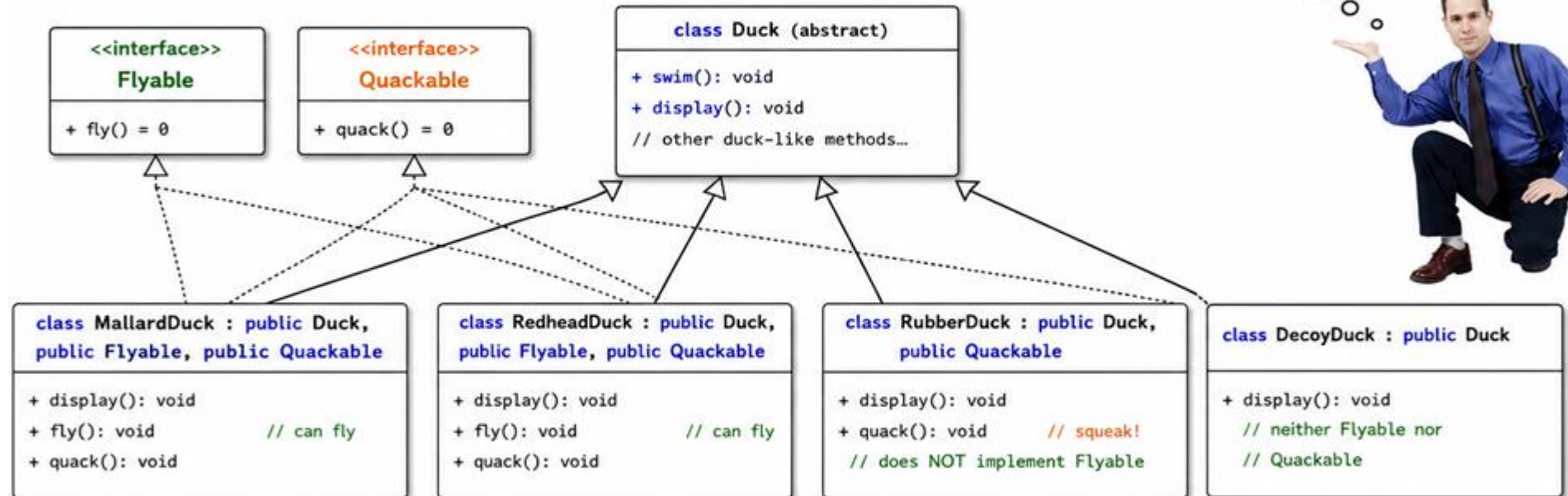
# Example: Sim

## Design Idea

Move fly() out of Duck and into a **Flyable** interface. Only ducks that can fly will implement it.

We can also make a **Quackable** interface so not all ducks have to quack.

I could take the fly() out of the Duck superclass, and make a **Flyable** interface with a fly() method. That way, only the ducks that are supposed to fly will implement that interface and have a fly() method... and I might as well make a **Quackable** too, since not all ducks can quack.



- Duck contains only common behaviors (swim, display, etc.).
- Flyable has fly(). Only ducks that can fly implement it.
- Quackable has quack(). Only ducks that can quack implement it.
- This avoids forcing every duck to fly or quack.

## Why is this better?

- ✓ More flexible
- ✓ Avoids forcing behavior on ducks that shouldn't have it
- ✓ Easier to add new kinds of ducks

# Flexibility

- A flexible design allows behavior to be changed or extended easily.
- Behavior implementations can be exchanged through a common interface.

# Coupling and Cohesion

- **Coupling:** describes how strongly one class/component depends on other classes/components.
  - **Design goal:** reduce unnecessary dependencies so changes remain localized.
- **Cohesion:** describes how closely the responsibilities of a class/module belong together.
  - **Design goal:** keep related responsibilities together and separate unrelated or changing responsibilities.
- Instead of placing all flying and quacking implementations inside Duck, the changing behaviors are separated into dedicated classes.

# Encapsulate What Varies

**Design Principle: Identify the aspects of your application that vary and separate them from what stays the same.**

- Take the parts that vary and “encapsulate” them, so that later you can alter or extend the parts that vary without affecting the rest of your code.
- Benefits:
  - Fewer unintended consequences from code changes and more flexibility in your systems!
- In SimUDuck, flying and quacking vary across ducks.
  - Move those behaviors into separate behavior classes.